ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

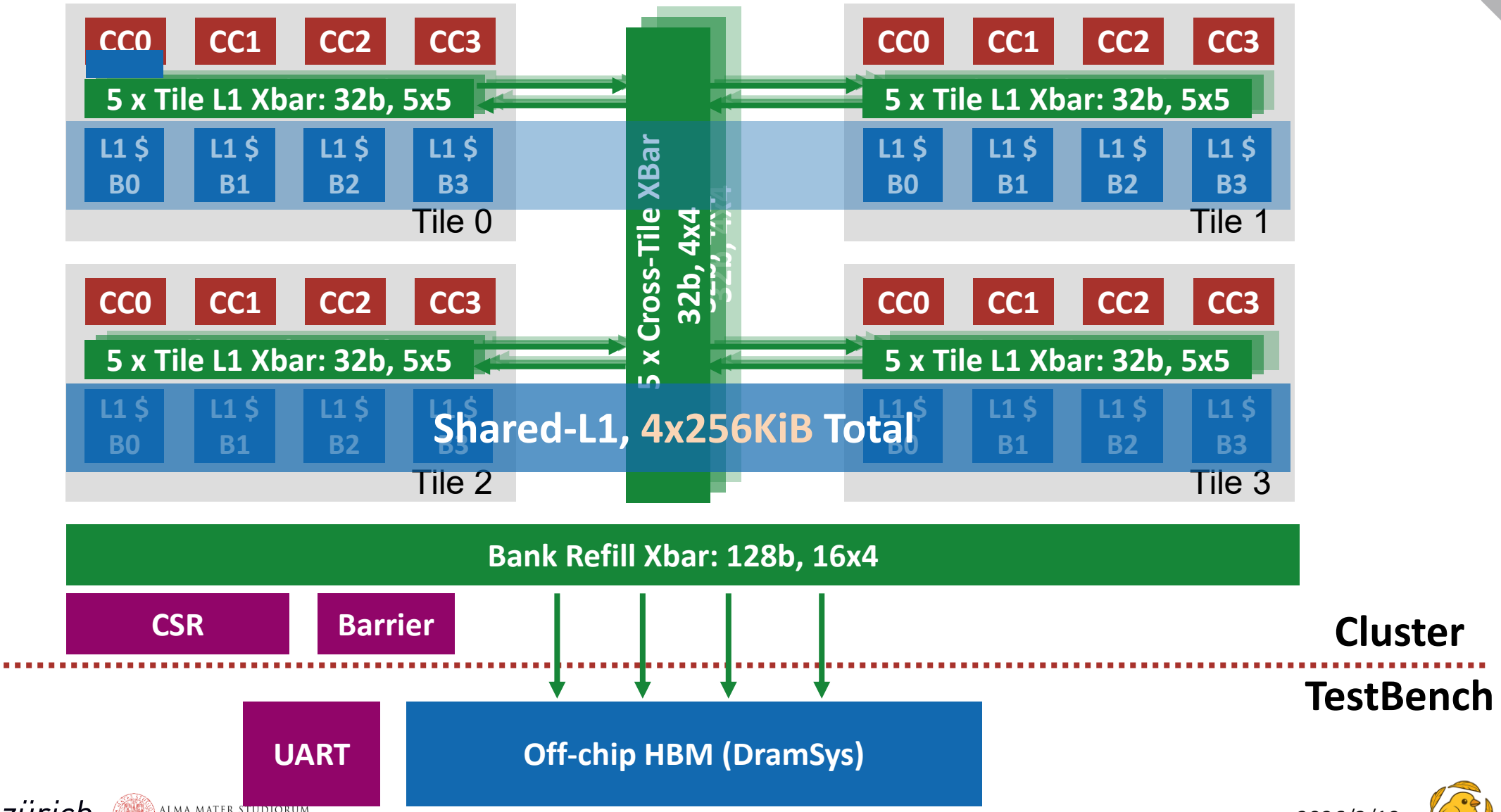
Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch

PULP Platform
Open Source Hardware, the way it should be!

@pulp_platform
pulp-platform.org
youtube.com/pulp_platform

Hardware Development (Cluster)

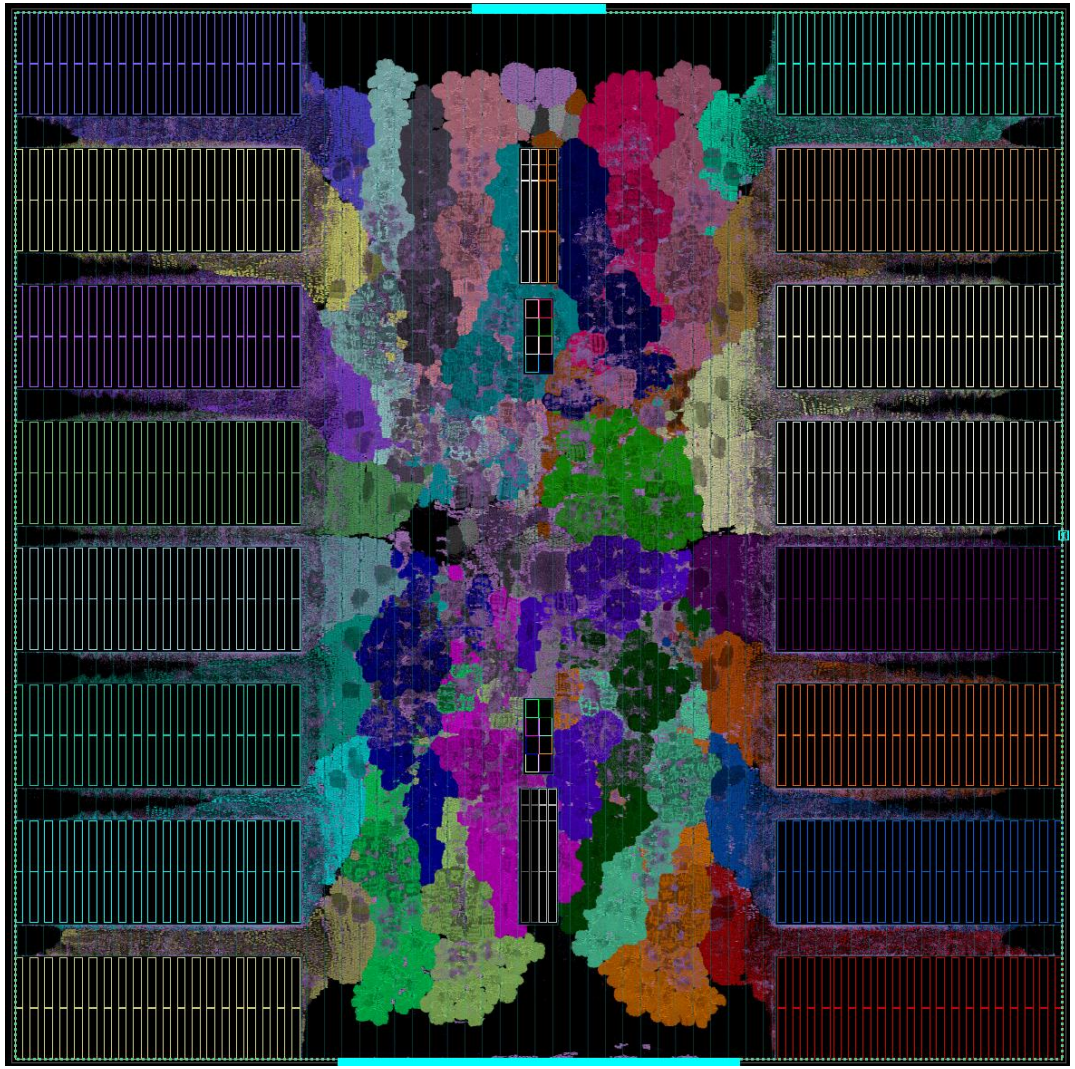


Hardware Development (Cluster)

- **Cache Partitioning**
  - Fixed some bugs in basic partitioning functionality
  - Test performance on matmul-64 kernel (algorithm not optimized for multi-tile)
    - Fully shared: 76% (hot cache)
    - Fully private: 97% (hot cache)
  - Reasons: access only to a few cache banks at a time in shared-cache mode
  - WIP
    - Half-half configuration
    - New linker address region for private region.
      Any data allocated in this region will be put into private partition when it exists.
      Then we can easily configure it from software

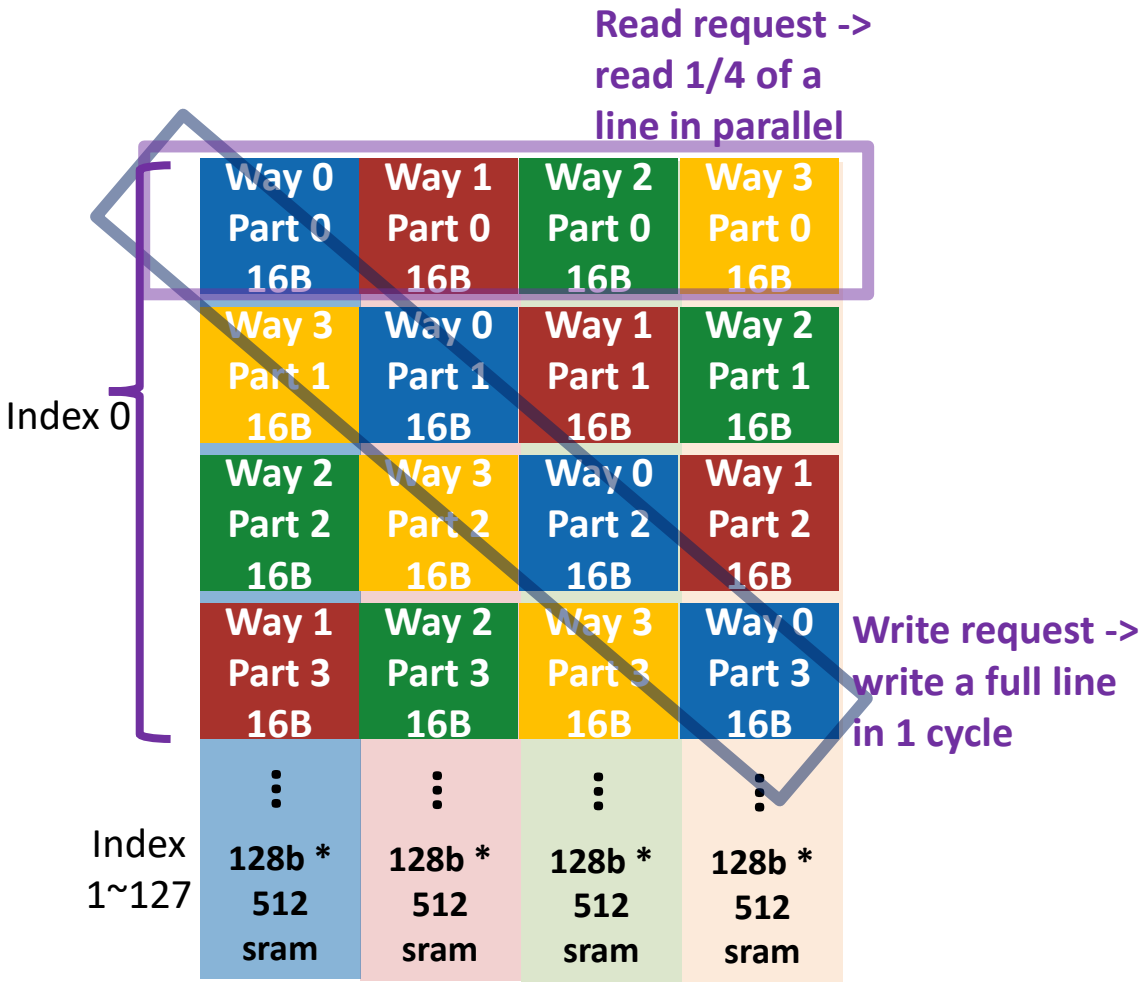
Hardware Development (Cluster)

- **Backend**
  - 4T16C, 4 DDR Channel
  - Old cache design
  - 12nm, 3mm-by-3mm die size
  - ~50% cell density, limited by SRAM
  - Critical path:
    - refill xbar rsp vld -> cache ctrl -> refill xbar rsp rdy (handshaking signals)
    - Add a cut on the response path, now rerunning till CTS



Hardware Development (L1 Cache)

- **Progress**
  - Passed all kernels except RLC (some corner cases)
  - Debugging undergoing
- **Benefits**
  - Significant area efficiency improvement in backend
    Original: One Way: 128x512b SRAM (not generatable) => 4 128x128b SRAMs. Need 512b data, extra address and control wires
    Folded: One Way: 512x128b SRAM. 4 times less SRAM macros. 128b data, no extra wires needed



Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in March. The progress on other side might be delayed
- **Diyou Side**
  - Finish cache partitioning
  - Finish backend exploration
  - Merge folded cache, collect benchmark and backend results
- **Zexin Side**
  - Finish the insitu cache controller debugging
  - Start the backend evaluation of the new cache controller design

Timeline

		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
System	Multi-Tile Scaling [RTL]													
	4/16-Group Scaling [GVSoC + RTL]													
L1 Access Latency	Hardware optimization (Cache & Interco) [RTL]													
	Cache partitioning [RTL]													
	WT-L1 by master student [RTL]													
Large NoC latency	Reduce per-hop latency [Physical]													
	Cut clk-domain (optional) [RTL + Physical]													
	NoC Topologies [RTL + GVSoC + Physical]													

Timeline

		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
Improve scalar core performance	Shared vector core [RTL + GVSoC]													
SW Kernel	Kernel optimization [SW]													
	Kernel development [SW]													
Report	Final Report and Benchmark													

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
  - **(Ongoing, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Ongoing, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Ongoing master thesis, now is working on the coherence between L1 caches
    => Currently plan to use hierarchical directory-based approach

L1 Access Latency

- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user

Thank you!